SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3: Articulation** (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	**Total Marks:**	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	S.Mahammed Nayeem	**Reg. No.:**	192425294
Section / Batch:	AI &ML/ 2024	**Date:**	06-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Code of Conduct

I S.Mahammed Nayeem / 192425294(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

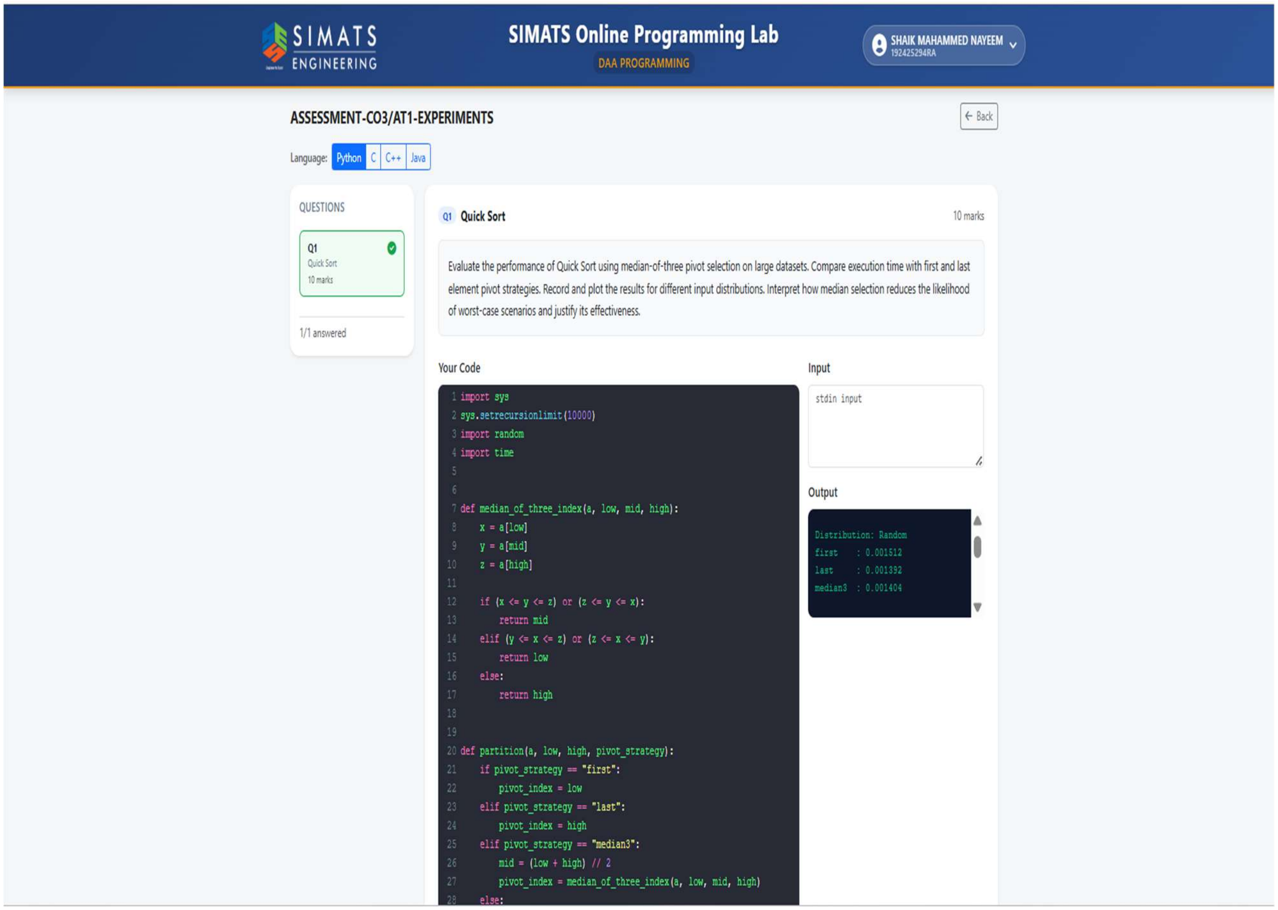
Signature of the Student: S.Mahammed Nayeem

PYTHON CODE FOR QUICK SORT :

```
import sys
sys.setrecursionlimit(10000)
import random
import time


def median_of_three_index(a, low, mid, high):
    x = a[low]
    y = a[mid]
    z = a[high]
    if (x <= y <= z) or (z <= y <= x):
        return mid
    elif (y <= x <= z) or (z <= x <= y):
        return low
    else:
        return high


def partition(a, low, high, strategy):
    if strategy == "first":
        pivot_index = low
    elif strategy == "last":
        pivot_index = high
    else:
        mid = (low + high) // 2
        pivot_index = median_of_three_index(a, low, mid, high)

    a[pivot_index], a[high] = a[high], a[pivot_index]
    pivot = a[high]

    i = low - 1
    for j in range(low, high):
```

```
        if a[j] <= pivot:
            i += 1
            a[i], a[j] = a[j], a[i]

    a[i + 1], a[high] = a[high], a[i + 1]
    return i + 1

def quick_sort(a, low, high, strategy):
    if low < high:
        pi = partition(a, low, high, strategy)
        quick_sort(a, low, pi - 1, strategy)
        quick_sort(a, pi + 1, high, strategy)

def measure(arr, strategy):
    temp = arr[:]
    start = time.perf_counter()
    quick_sort(temp, 0, len(temp) - 1, strategy)
    end = time.perf_counter()
    return end - start

n = int(input())

arr = [random.randint(1, 1000000) for _ in range(n)]

print("Distribution: Random")
print("first :", format(measure(arr, "first"), ".6f"))
print("last  :", format(measure(arr, "last"), ".6f"))
print("median:", format(measure(arr, "median"), ".6f"))
```

OUTPUT :



SIMATS
ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
SHAIK MAHAMMED NAYEEM
192425294RA
ASSESSMENT-CO3/AT1-EXPERIMENTS
Back
Language: Python C C++ Java
QUESTIONS
Q1
Quick Sort
10 marks
1/1 answered
Q1 Quick Sort
10 marks
Evaluate the performance of Quick Sort using median-of-three pivot selection on large datasets. Compare execution time with first and last element pivot strategies. Record and plot the results for different input distributions. Interpret how median selection reduces the likelihood of worst-case scenarios and justify its effectiveness.
Your Code
import sys
sys.setrecursionlimit(10000)
import random
import time
def median_of_three_index(a, low, mid, high):
x = a[low]
y = a[mid]
z = a[high]
if (x <= y <= z) or (z <= y <= x):
return mid
elif (y <= x <= z) or (z <= x <= y):
return low
else:
return high
def partition(a, low, high, pivot_strategy):
if pivot_strategy == "first":
pivot_index = low
elif pivot_strategy == "last":
pivot_index = high
elif pivot_strategy == "median3":
mid = (low + high) // 2
pivot_index = median_of_three_index(a, low, mid, high)
else:
Input
stdin input
Output
Distribution: Random
first : 0.001512
last : 0.001392
median3 : 0.001404

RUBRICS FOR CALCULATION

Experiments					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: ________________	Signature: ________________	Signature: ________________
Date: ____________________	Date: ____________________	Date: ____________________